Deep Learning

Advanced Topics: GANs, Transfer Learning & Continual Learning

Author: Eng. Carlos Andrés Sierra, M.Sc.
cavirguezs@udistrital.edu.co

Full-time Adjunct Professor
Computer Engineering Program
School of Engineering
Universidad Distrital Francisco José de Caldas

2026-I

Outline

1. Generative Adversarial Networks I
2. Generative Adversarial Networks II
3. Transfer Learning I: Pretrained Models & Few-Shot Classification
4. Transfer Learning II: Neural Style Transfer & Domain Adaptation
5. Online & Continual Learning

Outline

1. Generative Adversarial Networks I
2. Generative Adversarial Networks II
3. Transfer Learning I: Pretrained Models & Few-Shot Classification
4. Transfer Learning II: Neural Style Transfer & Domain Adaptation
5. Online & Continual Learning

Generative vs. Discriminative Models

Discriminative

- Model $P(y \mid \mathbf{x})$
- Map input to *label*
- Classes 1–6 focus here
- Examples: `ResNet`, `BERT`

Generative

- Model $P(\mathbf{x})$ or $P(\mathbf{x} \mid y)$
- Synthesise new samples
- Dense representation learning
- Examples: `VAE`, `GAN`, `Diffusion`

Why Generative Models?

- Data augmentation for rare classes
- Anomaly detection via *likelihood*
- Creative applications (*image/text/audio synthesis*)
- Bridge to Challenge 4 (`GAIL`): adversarial reward in `RL`

Generative vs. Discriminative Models

Discriminative

- Model $P(y \mid \mathbf{x})$
- Map input to *label*
- Classes 1–6 focus here
- Examples: `ResNet`, `BERT`

Generative

- Model $P(\mathbf{x})$ or $P(\mathbf{x} \mid y)$
- Synthesise new samples
- Dense representation learning
- Examples: `VAE`, `GAN`, `Diffusion`

Why Generative Models?

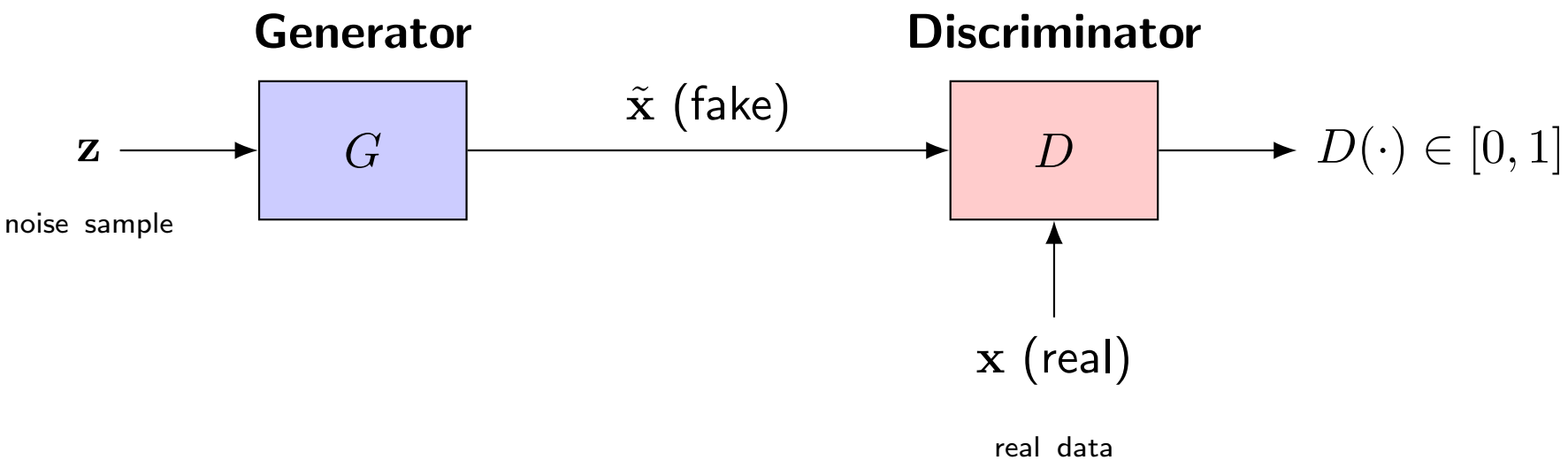
- Data augmentation for rare classes
- Anomaly detection via *likelihood*
- Creative applications (*image/text/audio synthesis*)
- Bridge to Challenge 4 (`GAIL`): adversarial reward in `RL`

The GAN Framework I

Goodfellow *et al.*, 2014

Two networks play a minimax game:

- `Generator` G: maps *noise* $\mathbf{z} \sim p_z(\mathbf{z})$ to fake data $\tilde{\mathbf{x}} = G(\mathbf{z})$
- `Discriminator` D: outputs probability that input is real



The GAN Framework II

Value Function

$$\min_G \max_D \; \mathbb{E}_{\mathbf{x}\sim p_{\text{data}}}[\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z}\sim p_z}[\log(1 - D(G(\mathbf{z})))]$$

- **Nash Equilibrium**: G generates the true data distribution $\Rightarrow$ $D(\mathbf{x}) = 0.5$ *everywhere*
- In practice: G maximises $\log D(G(\mathbf{z}))$ (non-saturating loss)

GAN Training Algorithm

1. Sample m real examples + m noise vectors

2. Update D: maximize discrimination

3. Update G: fool discriminator

Repeat until convergence

1. For each *training iteration*:
   - Sample m *real examples* $\{\mathbf{x}^{(i)}\}$ from p_{data}
   - Sample m *noise vectors* $\{\mathbf{z}^{(i)}\}$ from p_z
2. **Update** D: maximise $\frac{1}{m}\sum_i \left[\log D(\mathbf{x}^{(i)}) + \log(1 - D(G(\mathbf{z}^{(i)})))\right]$
3. **Update** G: minimise $\frac{1}{m}\sum_i \log(1 - D(G(\mathbf{z}^{(i)})))$ (equivalently maximise $\log D(G(\mathbf{z}))$)

GAN Training Challenges

Mode Collapse

G finds a small subset of outputs that reliably fool D and ignores the rest of the distribution. Generated images look similar to each other.

Non-Convergence

D wins early (loss $\to 0$) $\Rightarrow$ gradient signal to G vanishes. Training oscillates or diverges.

Common Remedies

- **Label smoothing**: *real labels* $= 0.9$, not 1.0
- **Feature matching**: train G to match D's intermediate statistics
- **Minibatch discrimination**: D penalises lack of diversity
- **Spectral normalisation**: constrain D's Lipschitz constant

GAN Training Challenges

Mode Collapse

G finds a small subset of outputs that reliably fool D and ignores the rest of the distribution. Generated images look similar to each other.

Non-Convergence

D wins early (loss $\to 0$) $\Rightarrow$ gradient signal to G vanishes. Training oscillates or diverges.

Common Remedies

- **Label smoothing**: *real labels* $= 0.9$, not 1.0
- **Feature matching**: train G to match D's intermediate statistics
- **Minibatch discrimination**: D penalises lack of diversity
- **Spectral normalisation**: constrain D's Lipschitz constant

Wasserstein GAN

Arjovsky *et al.*, 2017 — *Earth Mover Distance*

Replace the JS divergence with the Wasserstein-1 (*Earth Mover*) distance:

$$W(p_r, p_g) = \sup_{\|f\|_L \leq 1} \mathbb{E}_{x \sim p_r}[f(x)] - \mathbb{E}_{x \sim p_g}[f(x)]$$

WGAN Objective

Critic: $\max_{\|f\|_L \leq 1} \mathbb{E}[f(\mathbf{x})] - \mathbb{E}[f(G(\mathbf{z}))]$
Generator: $\min_G -\mathbb{E}[f(G(\mathbf{z}))]$

- Wasserstein distance always provides meaningful gradients — no more vanishing signal
- *Weight clipping* (original) or gradient penalty (WGAN-GP) enforces *Lipschitz* constraint
- WGAN-GP: most reliable GAN variant in practice

Wasserstein GAN

Arjovsky *et al.*, 2017 — *Earth Mover Distance*

Replace the JS divergence with the Wasserstein-1 (*Earth Mover*) distance:

$$W(p_r, p_g) = \sup_{\|f\|_L \leq 1} \mathbb{E}_{x \sim p_r}[f(x)] - \mathbb{E}_{x \sim p_g}[f(x)]$$

WGAN Objective

Critic: $\max_{\|f\|_L \leq 1} \mathbb{E}[f(\mathbf{x})] - \mathbb{E}[f(G(\mathbf{z}))]$
Generator: $\min_G -\mathbb{E}[f(G(\mathbf{z}))]$

- Wasserstein distance always provides meaningful gradients — no more vanishing signal
- *Weight clipping* (original) or gradient penalty (WGAN-GP) enforces *Lipschitz* constraint
- WGAN-GP: most reliable GAN variant in practice

Wasserstein GAN

Arjovsky *et al.*, 2017 — *Earth Mover Distance*

Replace the JS divergence with the Wasserstein-1 (*Earth Mover*) distance:

$$W(p_r, p_g) = \sup_{\|f\|_L \leq 1} \mathbb{E}_{x \sim p_r}[f(x)] - \mathbb{E}_{x \sim p_g}[f(x)]$$

WGAN Objective

Critic: $\max_{\|f\|_L \leq 1} \mathbb{E}[f(\mathbf{x})] - \mathbb{E}[f(G(\mathbf{z}))]$
Generator: $\min_G -\mathbb{E}[f(G(\mathbf{z}))]$

- Wasserstein distance always provides meaningful gradients — no more vanishing signal
- *Weight clipping* (original) or gradient penalty (WGAN-GP) enforces *Lipschitz* constraint
- WGAN-GP: most reliable GAN variant in practice

Evaluating GANs

Fréchet Inception Distance (FID)

$$\text{FID} = \|\boldsymbol{\mu}_r - \boldsymbol{\mu}_g\|^2 + \text{Tr}\left(\boldsymbol{\Sigma}_r + \boldsymbol{\Sigma}_g - 2\sqrt{\boldsymbol{\Sigma}_r \boldsymbol{\Sigma}_g}\right)$$

Compare Inception feature statistics of *real* vs. *generated* images. Lower is better.

Inception Score (IS)

$\text{IS} = \exp(\mathbb{E}_{\mathbf{x}}[D_{\text{KL}}(p(y|\mathbf{x})\|p(y))])$ Higher is better; measures *diversity* and *quality* simultaneously.

- FID is the standard benchmark (more reliable than IS)
- *Visual inspection* remains essential: FID alone can be gamed

Evaluating GANs

Fréchet Inception Distance (FID)

$$\text{FID} = \|\boldsymbol{\mu}_r - \boldsymbol{\mu}_g\|^2 + \text{Tr}\left(\boldsymbol{\Sigma}_r + \boldsymbol{\Sigma}_g - 2\sqrt{\boldsymbol{\Sigma}_r \boldsymbol{\Sigma}_g}\right)$$

Compare Inception feature statistics of *real* vs. *generated* images. Lower is better.

Inception Score (IS)

$\text{IS} = \exp(\mathbb{E}_{\mathbf{x}}\left[D_{\text{KL}}(p(y|\mathbf{x})\|p(y))\right])$ Higher is better; measures *diversity* and *quality* simultaneously.

- FID is the standard benchmark (more reliable than IS)
- *Visual inspection* remains essential: FID alone can be gamed

Evaluating GANs

Fréchet Inception Distance (FID)

$$\text{FID} = \|\boldsymbol{\mu}_r - \boldsymbol{\mu}_g\|^2 + \text{Tr}\left(\boldsymbol{\Sigma}_r + \boldsymbol{\Sigma}_g - 2\sqrt{\boldsymbol{\Sigma}_r \boldsymbol{\Sigma}_g}\right)$$

Compare Inception feature statistics of *real* vs. *generated* images. Lower is better.

Inception Score (IS)

$\text{IS} = \exp(\mathbb{E}_{\mathbf{x}}\left[D_{\text{KL}}(p(y|\mathbf{x})\|p(y))\right])$ Higher is better; measures *diversity* and *quality* simultaneously.

- FID is the standard benchmark (more reliable than IS)
- *Visual inspection* remains essential: FID alone can be gamed

Connection to Challenge 4 (GAIL)

Generative Adversarial Imitation Learning (Ho & Ermon, 2016)

GAIL applies the GAN framework to Reinforcement Learning:

- Generator $\equiv$ Policy π_θ: produces state-action trajectories
- Discriminator D_ϕ: distinguishes expert vs. policy trajectories
- **Adversarial reward**: $r_{\text{adv}}(s, a) = -\log(1 - D_\phi(s, a))$

Why GAN + RL?

GAIL learns to imitate without a reward function — the discriminator replaces the environment's reward. This directly links the GAN material in this class to Challenge 4.

Connection to Challenge 4 (GAIL)

Generative Adversarial Imitation Learning (Ho & Ermon, 2016)

GAIL applies the GAN framework to Reinforcement Learning:

- Generator $\equiv$ Policy π_θ: produces state-action trajectories
- Discriminator D_ϕ: distinguishes expert vs. policy trajectories
- **Adversarial reward**: $r_{\text{adv}}(s, a) = -\log(1 - D_\phi(s, a))$

Why GAN + RL?

GAIL learns to imitate without a reward function — the discriminator replaces the environment's reward. This directly links the GAN material in this class to Challenge 4.

Practical Example: Vanilla GAN on MNIST

- `Generator`: *noise* $\mathbf{z} \in \mathbb{R}^{100} \rightarrow$ FC layers $\rightarrow$ image 28×28
- `Discriminator`: *image* $\rightarrow$ FC layers $\rightarrow$ scalar
- **Track**: *discriminator loss*, *generator loss*, and `FID` every $1k$ steps
- **Observe**: training instability phases, mode collapse if it occurs
- **Experiment**: compare `JS`-loss vs. `WGAN-GP` loss on *stability*
- **Key insight**: curves that diverge in `JS-GAN` often converge with `WGAN-GP`

Practical Example: Vanilla GAN on MNIST

- `Generator`: *noise* $\mathbf{z} \in \mathbb{R}^{100} \rightarrow$ `FC` layers $\rightarrow$ image 28×28
- `Discriminator`: *image* $\rightarrow$ `FC` layers $\rightarrow$ scalar
- **Track**: *discriminator loss*, *generator loss*, and `FID` every $1k$ steps
- **Observe**: training instability phases, mode collapse if it occurs
- **Experiment**: compare `JS`-loss vs. `WGAN-GP` loss on *stability*
- **Key insight**: curves that diverge in `JS-GAN` often converge with `WGAN-GP`

Practical Example: Vanilla GAN on MNIST

- `Generator`: *noise* $\mathbf{z} \in \mathbb{R}^{100} \rightarrow$ `FC` layers $\rightarrow$ image 28×28
- `Discriminator`: *image* $\rightarrow$ `FC` layers $\rightarrow$ scalar
- **Track**: *discriminator loss*, *generator loss*, and `FID` every $1k$ steps
- **Observe**: training instability phases, mode collapse if it occurs
- **Experiment**: compare `JS`-loss vs. `WGAN-GP` loss on *stability*
- **Key insight**: curves that diverge in `JS-GAN` often converge with `WGAN-GP`

Outline

1. Generative Adversarial Networks I
2. Generative Adversarial Networks II
3. Transfer Learning I: Pretrained Models & Few-Shot Classification
4. Transfer Learning II: Neural Style Transfer & Domain Adaptation
5. Online & Continual Learning

DCGAN: Deep Convolutional GAN

Radford *et al.*, 2015

Replace fully-connected layers with convolutional layers for *image synthesis*.

Generator

- FC $\rightarrow$ reshape to 4×4
- Transposed convolutions (*upsample*)
- BatchNorm after each conv
- ReLU *hidden*, tanh output

Discriminator

- Strided convolutions (*downsample*)
- LeakyReLU (slope 0.2)
- BatchNorm (except *first/last*)
- Sigmoid output

DCGAN Rules of Thumb

No *pooling* (use strided conv), BatchNorm everywhere except input and output layers.

DCGAN: Deep Convolutional GAN

Radford *et al.*, 2015

Replace fully-connected layers with convolutional layers for *image synthesis*.

Generator

- FC $\rightarrow$ reshape to 4×4
- Transposed convolutions (*upsample*)
- BatchNorm after each conv
- ReLU *hidden*, tanh output

Discriminator

- Strided convolutions (*downsample*)
- LeakyReLU (slope 0.2)
- BatchNorm (except *first/last*)
- Sigmoid output

DCGAN Rules of Thumb

No *pooling* (use strided conv), BatchNorm everywhere except input and output layers.

DCGAN: Deep Convolutional GAN

Radford *et al.*, 2015

Replace fully-connected layers with convolutional layers for *image synthesis*.

Generator

- FC $\rightarrow$ reshape to 4×4
- Transposed convolutions (*upsample*)
- BatchNorm after each conv
- ReLU *hidden*, tanh output

Discriminator

- Strided convolutions (*downsample*)
- LeakyReLU (slope 0.2)
- BatchNorm (except *first/last*)
- Sigmoid output

DCGAN Rules of Thumb

No *pooling* (use strided conv), BatchNorm everywhere except input and output layers.

Conditional GAN (cGAN)

Mirza & Osindero, 2014

Condition both G and D on auxiliary information $\mathbf{y}$ (*class label*, *text*, etc.):

$$\min_G \max_D \; \mathbb{E}[\log D(\mathbf{x} \mid \mathbf{y})] + \mathbb{E}[\log(1 - D(G(\mathbf{z} \mid \mathbf{y})))]$$

- $\mathbf{y}$ concatenated with $\mathbf{z}$ in G and with feature maps in D
- Enables class-conditional generation: generate a specific digit, object, etc.
- Foundation for image-to-image translation: condition on an entire *input image*

Conditional GAN (cGAN)

Mirza & Osindero, 2014

Condition both G and D on auxiliary information $\mathbf{y}$ (*class label*, *text*, etc.):

$$\min_G \max_D \mathbb{E}[\log D(\mathbf{x} \mid \mathbf{y})] + \mathbb{E}[\log(1 - D(G(\mathbf{z} \mid \mathbf{y})))]$$

- $\mathbf{y}$ concatenated with $\mathbf{z}$ in G and with feature maps in D
- Enables class-conditional generation: generate a specific digit, object, etc.
- Foundation for image-to-image translation: condition on an entire *input image*

Pix2Pix: Image-to-Image Translation

Isola *et al.*, 2017

Learn a mapping from input image $\mathbf{x}$ to output image $\mathbf{y}$ (paired training data required):

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{cGAN}}(G, D) + \lambda\, \mathcal{L}_{L_1}(G)$$

- The L_1 term forces pixel-level faithfulness to the input
- D operates on image patches (`PatchGAN`) rather than *full images*
- **Applications**: sketch $\rightarrow$ photo, semantic map $\rightarrow$ scene, day $\rightarrow$ night, grayscale $\rightarrow$ colour

Pix2Pix: Image-to-Image Translation

Isola *et al.*, 2017

Learn a mapping from input image $\mathbf{x}$ to output image $\mathbf{y}$ (paired training data required):

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{cGAN}}(G, D) + \lambda\, \mathcal{L}_{L_1}(G)$$

- The L_1 term forces pixel-level faithfulness to the input
- D operates on image patches (`PatchGAN`) rather than *full images*
- **Applications**: sketch $\rightarrow$ photo, semantic map $\rightarrow$ scene, day $\rightarrow$ night, grayscale $\rightarrow$ colour

Pix2Pix: Image-to-Image Translation

Isola *et al.*, 2017

Learn a mapping from input image $\mathbf{x}$ to output image $\mathbf{y}$ (paired training data required):

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{cGAN}}(G, D) + \lambda\, \mathcal{L}_{L_1}(G)$$

- The L_1 term forces pixel-level faithfulness to the input
- D operates on image patches (`PatchGAN`) rather than *full images*
- **Applications**: sketch $\rightarrow$ photo, semantic map $\rightarrow$ scene, day $\rightarrow$ night, grayscale $\rightarrow$ colour

CycleGAN: Unpaired Image Translation

Zhu *et al.*, 2017

No paired training data. Two generators $G_{X \to Y}$ and $F_{Y \to X}$ with cycle-consistency loss:

$$\mathcal{L}_{\text{cyc}} = \mathbb{E}[\|F(G(\mathbf{x})) - \mathbf{x}\|_1] + \mathbb{E}[\|G(F(\mathbf{y})) - \mathbf{y}\|_1]$$

- Cycle-consistency: one translation followed by its inverse recovers the original
- **Applications**: photo ↔ painting, horse ↔ zebra, summer ↔ winter
- **Connection to Challenge 7 Part B**: the neural style transfer operation is related to — but *distinct from* — CycleGAN

CycleGAN: Unpaired Image Translation

Zhu *et al.*, 2017

No paired training data. Two generators $G_{X \to Y}$ and $F_{Y \to X}$ with cycle-consistency loss:

$$\mathcal{L}_{\text{cyc}} = \mathbb{E}[\|F(G(\mathbf{x})) - \mathbf{x}\|_1] + \mathbb{E}[\|G(F(\mathbf{y})) - \mathbf{y}\|_1]$$

- Cycle-consistency: one translation followed by its inverse recovers the original
- **Applications**: photo $\leftrightarrow$ painting, horse $\leftrightarrow$ zebra, summer $\leftrightarrow$ winter
- **Connection to Challenge 7 Part B**: the neural style transfer operation is related to — but *distinct from* — CycleGAN

CycleGAN: Unpaired Image Translation

Zhu *et al.*, 2017

No paired training data. Two generators $G_{X \to Y}$ and $F_{Y \to X}$ with cycle-consistency loss:

$$\mathcal{L}_{\text{cyc}} = \mathbb{E}[\|F(G(\mathbf{x})) - \mathbf{x}\|_1] + \mathbb{E}[\|G(F(\mathbf{y})) - \mathbf{y}\|_1]$$

- Cycle-consistency: one translation followed by its inverse recovers the original
- **Applications**: photo $\leftrightarrow$ painting, horse $\leftrightarrow$ zebra, summer $\leftrightarrow$ winter
- **Connection to Challenge 7 Part B**: the neural style transfer operation is related to — but *distinct from* — CycleGAN

Progressive and Style GANs

ProGAN (Karras *et al.*, 2018)

Train at progressively larger resolutions: $4\times4 \to 8\times8 \to \cdots \to 1024\times1024$. Add new Conv layers as *resolution* increases. Stable training, photorealistic faces.

StyleGAN (Karras *et al.*, 2019)

- Latent code $\mathbf{z} \to$ mapping network $\to \mathbf{w} \in \mathcal{W}$
- $\mathbf{w}$ controls per-layer style via Adaptive Instance Normalisation (AdaIN)
- Style mixing: interpolate *styles* at *different layers* for controllable synthesis
- Unprecedented photorealism; enabled *deepfake* research and detection

Progressive and Style GANs

ProGAN (Karras *et al.*, 2018)

Train at progressively larger resolutions: $4\times4 \to 8\times8 \to \cdots \to 1024\times1024$. Add new Conv layers as *resolution* increases. Stable training, photorealistic faces.

StyleGAN (Karras *et al.*, 2019)

- Latent code $\mathbf{z} \to$ mapping network $\to \mathbf{w} \in \mathcal{W}$
- $\mathbf{w}$ controls per-layer style via Adaptive Instance Normalisation (AdaIN)
- Style mixing: interpolate *styles* at *different layers* for controllable synthesis
- Unprecedented photorealism; enabled *deepfake* research and detection

Diffusion Models (*Modern Context*)

Denoising Diffusion Probabilistic Models (DDPM)

A newer paradigm that outperforms GANs on image quality (Ho *et al.*, 2020). Gradually add Gaussian noise to data, then learn to reverse the *diffusion*.

- Training objective: predict the added noise at each step
- Stable training (no adversarial instability)
- Stable DDPM: Denoising Diffusion Probabilistic Models
- Trade-off: slow sampling (many steps) vs. GANs (one step)

Context for This Course

GANs remain the conceptual foundation and are central to Challenge 4 (GAIL). Diffusion models are the frontier — referenced here as *motivation* for *further study*.

Diffusion Models (*Modern Context*)

Denoising Diffusion Probabilistic Models (DDPM)

A newer paradigm that outperforms GANs on image quality (Ho *et al.*, 2020). Gradually add Gaussian noise to data, then learn to reverse the *diffusion*.

- Training objective: predict the added noise at each step
- Stable training (no adversarial instability)
- Enables: DALL-E 2, Stable Diffusion, Midjourney
- **Trade-off**: better sample quality; slower inference than GAN

Context for This Course

GANs remain the conceptual foundation and are central to Challenge 4 (GAIL). Diffusion models are the frontier — referenced here as *motivation* for *further study*.

Diffusion Models (*Modern Context*)

Denoising Diffusion Probabilistic Models (DDPM)

A newer paradigm that outperforms GANs on image quality (Ho *et al.*, 2020). Gradually add Gaussian noise to data, then learn to reverse the *diffusion*.

- Training objective: predict the added noise at each step
- Stable training (no adversarial instability)
- Enables: DALL-E 2, Stable Diffusion, Midjourney
- **Trade-off**: better sample quality; slower inference than GAN

Context for This Course

GANs remain the conceptual foundation and are central to Challenge 4 (GAIL). Diffusion models are the frontier — referenced here as *motivation* for *further study*.

Diffusion Models (*Modern Context*)

Denoising Diffusion Probabilistic Models (DDPM)

A newer paradigm that outperforms GANs on image quality (Ho *et al.*, 2020). Gradually add Gaussian noise to data, then learn to reverse the *diffusion*.

- Training objective: predict the added noise at each step
- Stable training (no adversarial instability)
- Enables: DALL-E 2, Stable Diffusion, Midjourney
- **Trade-off**: better sample quality; slower inference than GAN

Context for This Course

GANs remain the conceptual foundation and are central to Challenge 4 (GAIL). Diffusion models are the frontier — referenced here as *motivation* for *further study*.

Diffusion Models (*Modern Context*)

Denoising Diffusion Probabilistic Models (DDPM)

A newer paradigm that outperforms GANs on image quality (Ho *et al.*, 2020). Gradually add Gaussian noise to data, then learn to reverse the *diffusion*.

- Training objective: predict the added noise at each step
- Stable training (no adversarial instability)
- Enables: DALL-E 2, Stable Diffusion, Midjourney
- **Trade-off**: better sample quality; slower inference than GAN

Context for This Course

GANs remain the conceptual foundation and are central to Challenge 4 (GAIL). Diffusion models are the frontier — referenced here as *motivation* for *further study*.

GAN Evaluation Checklist

- **Loss curves**: generator and discriminator should be roughly balanced
- FID: compute on held-out real set vs. 10k generated samples
- **Visual diversity**: plot a grid of 8×8 generated samples; check for *mode collapse*
- **Discriminator accuracy**: above 85% early $\Rightarrow$ G is not learning *fast enough*
- **Interpolation**: linear interpolation in $\mathbf{z}$-space should produce *smooth transitions*
- **For Challenge 4 (GAIL)**: discriminator accuracy over training reveals how fast the policy matches the *expert occupancy measure*

GAN Evaluation Checklist

- **Loss curves**: generator and discriminator should be roughly balanced
- FID: compute on held-out real set vs. 10k generated samples
- **Visual diversity**: plot a grid of 8×8 generated samples; check for *mode collapse*
- **Discriminator accuracy**: above 85% early $\Rightarrow G$ is not learning *fast enough*
- **Interpolation**: linear interpolation in $\mathbf{z}$-space should produce *smooth transitions*
- **For Challenge 4 (GAIL)**: discriminator accuracy over training reveals how fast the policy matches the *expert occupancy measure*

GAN Evaluation Checklist

- **Loss curves**: generator and discriminator should be roughly balanced
- FID: compute on held-out real set vs. 10k generated samples
- **Visual diversity**: plot a grid of 8×8 generated samples; check for *mode collapse*
- **Discriminator accuracy**: above 85% early $\Rightarrow G$ is not learning *fast enough*
- **Interpolation**: linear interpolation in $\mathbf{z}$-space should produce *smooth transitions*
- **For Challenge 4 (GAIL)**: discriminator accuracy over training reveals how fast the policy matches the *expert occupancy measure*

GAN Evaluation Checklist

- **Loss curves**: generator and discriminator should be roughly balanced
- FID: compute on held-out real set vs. 10k generated samples
- **Visual diversity**: plot a grid of 8×8 generated samples; check for *mode collapse*
- **Discriminator accuracy**: above 85% early $\Rightarrow G$ is not learning *fast enough*
- **Interpolation**: linear interpolation in $\mathbf{z}$-space should produce *smooth transitions*
- **For Challenge 4 (GAIL)**: discriminator accuracy over training reveals how fast the policy matches the *expert occupancy measure*

GAN Evaluation Checklist

- **Loss curves**: generator and discriminator should be roughly balanced
- FID: compute on held-out real set vs. 10k generated samples
- **Visual diversity**: plot a grid of 8×8 generated samples; check for *mode collapse*
- **Discriminator accuracy**: above 85% early $\Rightarrow G$ is not learning *fast enough*
- **Interpolation**: linear interpolation in $\mathbf{z}$-space should produce *smooth transitions*
- **For Challenge 4 (GAIL)**: discriminator accuracy over training reveals how fast the policy matches the *expert occupancy measure*

GAN Evaluation Checklist

- **Loss curves**: generator and discriminator should be roughly balanced
- FID: compute on held-out real set vs. 10k generated samples
- **Visual diversity**: plot a grid of 8×8 generated samples; check for *mode collapse*
- **Discriminator accuracy**: above 85% early $\Rightarrow G$ is not learning *fast enough*
- **Interpolation**: linear interpolation in $\mathbf{z}$-space should produce *smooth transitions*
- **For Challenge 4 (GAIL)**: discriminator accuracy over training reveals how fast the policy matches the *expert occupancy measure*

Practical Example: DCGAN on CIFAR-10

- `Generator`: noise $\mathbf{z} \in \mathbb{R}^{128} \to$ reshape $\to 4\times4 \to$ 4 `ConvTranspose` blocks $\to 32\times32\times3$
- `Discriminator`: $32\times32\times3 \to$ 4 *strided* `Conv` blocks $\to$ scalar
- **Experiment A**: `DCGAN` with `BCE` loss; record `FID` at *intervals*
- **Experiment B**: swap to `WGAN-GP` loss; compare stability
- **Experiment C**: add *class labels* to produce a `cGAN`
- **Expected**: `WGAN-GP` $<$ `FID` than standard `DCGAN`; `cGAN` generates recognisable class-conditional samples

Practical Example: DCGAN on CIFAR-10

- `Generator`: noise $\mathbf{z} \in \mathbb{R}^{128} \to$ reshape $\to 4\times4 \to$ 4 `ConvTranspose` blocks $\to 32\times32\times3$
- `Discriminator`: $32\times32\times3 \to$ 4 *strided* `Conv` blocks $\to$ scalar
- **Experiment A**: `DCGAN` with `BCE` loss; record `FID` at *intervals*
- **Experiment B**: swap to `WGAN-GP` loss; compare stability
- **Experiment C**: add *class labels* to produce a `cGAN`
- **Expected**: `WGAN-GP` $<$ `FID` than standard `DCGAN`; `cGAN` generates recognisable class-conditional samples

Practical Example: DCGAN on CIFAR-10

- `Generator`: noise $\mathbf{z} \in \mathbb{R}^{128} \rightarrow$ reshape $\rightarrow 4\times4 \rightarrow$ 4 `ConvTranspose` blocks $\rightarrow 32\times32\times3$
- `Discriminator`: $32\times32\times3 \rightarrow$ 4 *strided* `Conv` blocks $\rightarrow$ scalar
- **Experiment A**: `DCGAN` with `BCE` loss; record `FID` at *intervals*
- **Experiment B**: swap to `WGAN-GP` loss; compare stability
- **Experiment C**: add *class labels* to produce a `cGAN`
- **Expected**: `WGAN-GP` $<$ `FID` than standard `DCGAN`; `cGAN` generates recognisable class-conditional samples

Outline

1. Generative Adversarial Networks I
2. Generative Adversarial Networks II
3. Transfer Learning I: Pretrained Models & Few-Shot Classification
4. Transfer Learning II: Neural Style Transfer & Domain Adaptation
5. Online & Continual Learning

The Transfer Learning Paradigm

Core Idea

Reuse representations learned on a large *source task* (e.g. `ImageNet`, 1.2M images, 1000 classes) to solve a target task with much less data.

- **Why it works**: *low-level layers* learn universal features (edges, textures, shapes) that transfer across vision tasks
- **Data efficiency**: reach high accuracy with 50–500 examples per class vs. requiring *tens of thousands* from scratch
- **Compute efficiency**: *fine-tuning* takes *minutes to hours*; *training from scratch* takes *days to weeks* on a GPU cluster

The Transfer Learning Paradigm

Core Idea

Reuse representations learned on a large *source task* (e.g. `ImageNet`, 1.2M images, 1000 classes) to solve a target task with much less data.

- **Why it works**: *low-level layers* learn universal features (edges, textures, shapes) that transfer across vision tasks
- **Data efficiency**: reach high accuracy with 50–500 examples per class vs. requiring *tens of thousands* from scratch
- **Compute efficiency**: *fine-tuning* takes *minutes to hours*; *training from scratch* takes *days to weeks* on a `GPU` cluster

What is Transferred?

Layer-by-Layer Analysis

- **Early layers (layers 1–3)**: Gabor-like edge detectors, colour blobs — highly transferable across all *visual tasks*
- **Middle layers (layers 4–6)**: texture patterns, object parts — moderately transferable; more *task-specific*
- **Top layers (layers 7+)**: class-specific concept detectors — least transferable; replace for *new tasks*

- **Visualisation**: maximise activation of individual neurons (feature visualisation)
- **Practical rule**: the more different source and *target domains* are, the fewer layers to *transfer*

What is Transferred?

Layer-by-Layer Analysis

- **Early layers (layers 1–3)**: Gabor-like edge detectors, colour blobs — highly transferable across all *visual tasks*
- **Middle layers (layers 4–6)**: texture patterns, object parts — moderately transferable; more *task-specific*
- **Top layers (layers 7+)**: class-specific concept detectors — least transferable; replace for *new tasks*

- **Visualisation**: maximise activation of individual neurons (feature visualisation)
- **Practical rule**: the more different source and *target domains* are, the fewer layers to *transfer*

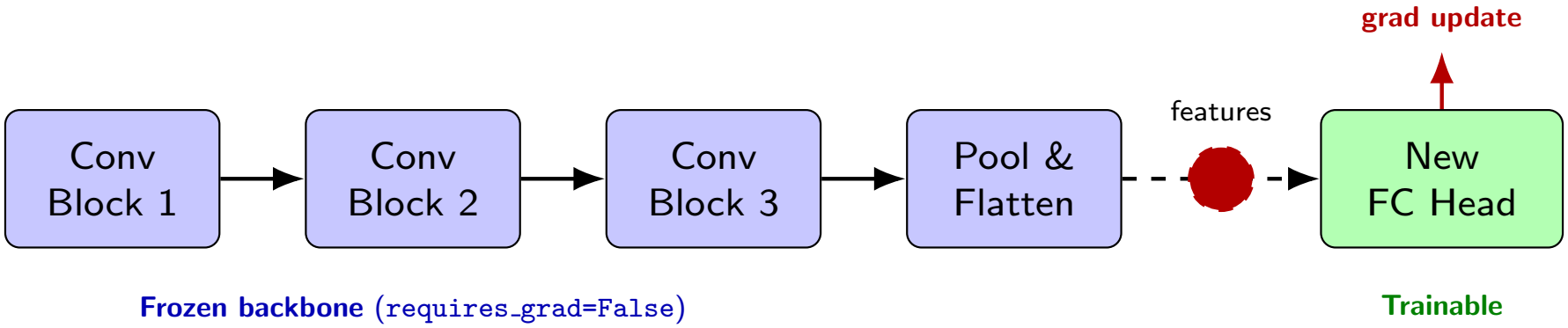
What is Transferred?

Layer-by-Layer Analysis

- **Early layers (layers 1–3)**: Gabor-like edge detectors, colour blobs — highly transferable across all *visual tasks*
- **Middle layers (layers 4–6)**: texture patterns, object parts — moderately transferable; more *task-specific*
- **Top layers (layers 7+)**: class-specific concept detectors — least transferable; replace for *new tasks*

- **Visualisation**: maximise activation of individual neurons (feature visualisation)
- **Practical rule**: the more different source and *target domains* are, the fewer layers to *transfer*

Two Transfer Strategies I

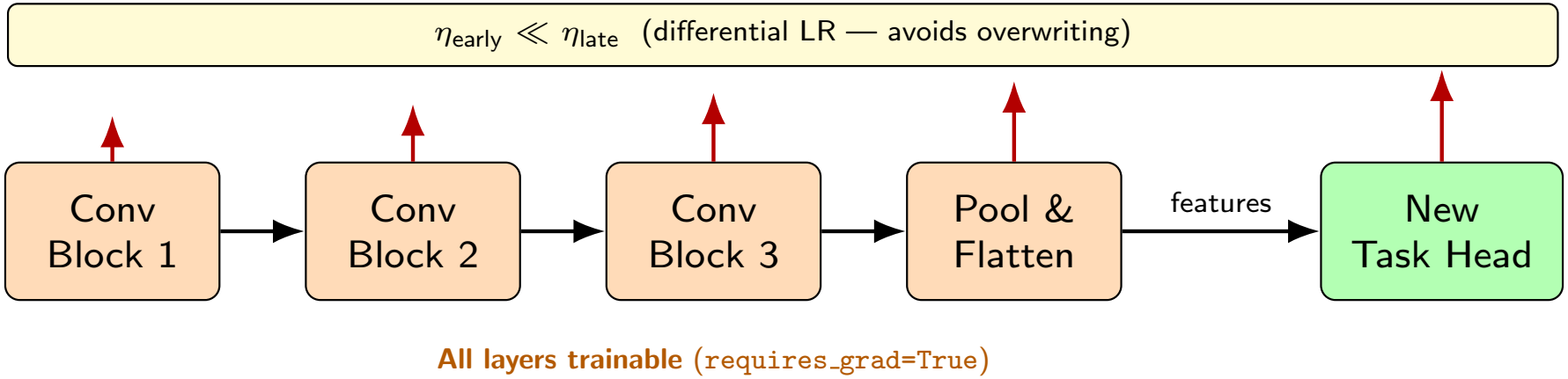


Strategy 1 — Feature Extraction (*Frozen Backbone*)

Freeze all pretrained layers; replace and train only the new classification head.

- Fast: only *head* is updated
- Works well when source and target domains are similar
- Recommended when target dataset is small (< 1k images/class)

Two Transfer Strategies II



Strategy 2 — Fine-Tuning

Unfreeze some or all pretrained layers; train with a small learning rate (e.g. 10^{-5}–10^{-4}) to avoid *catastrophic overwriting*.

- Slower; requires more *data*; risks overfitting on small datasets
- Best results when source and target domains differ substantially

When to Use Which Strategy

	Similar Domain	**Different Domain**
Small dataset	Feature extraction (high accuracy)	Feature extraction (reasonable accuracy)
Large dataset	Fine-tune few top layers	Fine-tune full network (with careful LR)

Practical Guideline for Challenge 7

Start with frozen backbone; if validation accuracy *plateaus*, progressively unfreeze *top residual blocks*. Use smaller LR for *deeper layers* (differential LR).

When to Use Which Strategy

	Similar Domain	**Different Domain**
Small dataset	Feature extraction (high accuracy)	Feature extraction (reasonable accuracy)
Large dataset	Fine-tune few top layers	Fine-tune full network (with careful LR)

Practical Guideline for Challenge 7

Start with frozen backbone; if validation accuracy *plateaus*, progressively unfreeze *top residual blocks*. Use smaller `LR` for *deeper layers* (differential LR).

Popular Pretrained Backbones

- `ResNet-50` / `ResNet-101`: standard baseline; strong accuracy–speed trade-off
- `EfficientNet-B0` **to** `B7`: *compound scaling*; mobile-friendly; comparable or better than `ResNet` with fewer parameters
- `MobileNet-V3`: designed for *edge devices*; 3–5× faster than `ResNet-50`
- `ViT` **(Vision Transformer)**: *patch-based*; excels when data is large; less inductive bias
- `CLIP` **(OpenAI)**: *text+image pretraining*; zero-shot classification possible

For Challenge 7

Use `ResNet-50` as the primary backbone; compare with `EfficientNet-B0` as an alternative. Both available in `torchvision.models` with `IMAGENET1K_V2` weights.

Popular Pretrained Backbones

- `ResNet-50` / `ResNet-101`: standard baseline; strong accuracy–speed trade-off
- `EfficientNet-B0` **to** `B7`: *compound scaling*; mobile-friendly; comparable or better than `ResNet` with fewer parameters
- `MobileNet-V3`: designed for *edge devices*; 3–5× faster than `ResNet-50`
- `ViT` **(Vision Transformer)**: *patch-based*; excels when data is large; less inductive bias
- `CLIP` **(OpenAI)**: *text+image pretraining*; zero-shot classification possible

For Challenge 7

Use `ResNet-50` as the primary backbone; compare with `EfficientNet-B0` as an alternative. Both available in `torchvision.models` with `IMAGENET1K_V2` weights.

Few-Shot Learning

Setup

Learn a new task from only K *labelled examples* per class ($K = 1, 5, 10, 50$). This is impossible for training from scratch but feasible with pretrained backbones.

Prototypical Networks (Snell *et al.*, 2017)

Classify by distance to class prototypes in *embedding space*:

$$\mathbf{c}_k = \frac{1}{|S_k|} \sum_{(\mathbf{x},y)\in S_k} f_\theta(\mathbf{x}), \qquad p(y = k \mid \mathbf{x}) = \frac{\exp(-d(f_\theta(\mathbf{x}), \mathbf{c}_k))}{\sum_j \exp(-d(f_\theta(\mathbf{x}), \mathbf{c}_j))}$$

- f_θ is the pretrained backbone (*fixed*)
- No *fine-tuning* needed: the *frozen features* are already discriminative

Few-Shot Learning

Setup

Learn a new task from only K *labelled examples* per class ($K = 1, 5, 10, 50$). This is impossible for training from scratch but feasible with pretrained backbones.

`Prototypical Networks` (Snell *et al.*, 2017)

Classify by distance to class prototypes in *embedding space*:

$$\mathbf{c}_k = \frac{1}{|S_k|} \sum_{(\mathbf{x},y)\in S_k} f_\theta(\mathbf{x}), \qquad p(y = k \mid \mathbf{x}) = \frac{\exp(-d(f_\theta(\mathbf{x}), \mathbf{c}_k))}{\sum_j \exp(-d(f_\theta(\mathbf{x}), \mathbf{c}_j))}$$

- f_θ is the pretrained backbone (*fixed*)
- No *fine-tuning* needed: the *frozen features* are already discriminative

Few-Shot Learning

Setup

Learn a new task from only K *labelled examples* per class ($K = 1, 5, 10, 50$). This is impossible for training from scratch but feasible with pretrained backbones.

`Prototypical Networks` (Snell *et al.*, 2017)

Classify by distance to class prototypes in *embedding space*:

$$\mathbf{c}_k = \frac{1}{|S_k|} \sum_{(\mathbf{x},y)\in S_k} f_\theta(\mathbf{x}), \qquad p(y = k \mid \mathbf{x}) = \frac{\exp(-d(f_\theta(\mathbf{x}), \mathbf{c}_k))}{\sum_j \exp(-d(f_\theta(\mathbf{x}), \mathbf{c}_j))}$$

- f_θ is the pretrained backbone (*fixed*)
- No *fine-tuning* needed: the *frozen features* are already discriminative

ImageNet Preprocessing Standard

Required for All Pretrained PyTorch Models

$$\text{Normalise}(\mathbf{x}) = \frac{\mathbf{x} - \boldsymbol{\mu}}{\boldsymbol{\sigma}}$$

$$\boldsymbol{\mu} = [0.485,\ 0.456,\ 0.406], \quad \boldsymbol{\sigma} = [0.229,\ 0.224,\ 0.225]$$

- Resize to 224×224 (for `ResNet`, `EfficientNet`, `ViT` families)
- Apply the same normalisation to *target domain* images — a common mistake in Challenge 7 is forgetting this step for cartoon/sketch images
- Training augmentations (random crop, flip, colour jitter) are applied only to source images
- Use `torchvision.transforms.v2` or `albumentations` for efficient augmentation pipelines

ImageNet Preprocessing Standard

Required for All Pretrained PyTorch Models

$$\text{Normalise}(\mathbf{x}) = \frac{\mathbf{x} - \boldsymbol{\mu}}{\boldsymbol{\sigma}}$$

$$\boldsymbol{\mu} = [0.485,\ 0.456,\ 0.406], \quad \boldsymbol{\sigma} = [0.229,\ 0.224,\ 0.225]$$

- Resize to 224×224 (for `ResNet`, `EfficientNet`, `ViT` families)
- Apply the same normalisation to *target domain* images — a common mistake in Challenge 7 is forgetting this step for cartoon/sketch images
- Training augmentations: *random horizontal flip*, *random crop*, *color jitter*
- Validation/test: deterministic *centre crop* at 224×224; no augmentation

ImageNet Preprocessing Standard

Required for All Pretrained PyTorch Models

$$\text{Normalise}(\mathbf{x}) = \frac{\mathbf{x} - \boldsymbol{\mu}}{\boldsymbol{\sigma}}$$

$$\boldsymbol{\mu} = [0.485,\ 0.456,\ 0.406], \quad \boldsymbol{\sigma} = [0.229,\ 0.224,\ 0.225]$$

- Resize to 224×224 (for `ResNet`, `EfficientNet`, `ViT` families)
- Apply the same normalisation to *target domain* images — a common mistake in Challenge 7 is forgetting this step for cartoon/sketch images
- Training augmentations: *random horizontal flip*, *random crop*, *color jitter*
- Validation/test: deterministic *centre crop* at 224×224; no augmentation

ImageNet Preprocessing Standard

Required for All Pretrained PyTorch Models

$$\text{Normalise}(\mathbf{x}) = \frac{\mathbf{x} - \boldsymbol{\mu}}{\boldsymbol{\sigma}}$$

$$\boldsymbol{\mu} = [0.485,\ 0.456,\ 0.406], \quad \boldsymbol{\sigma} = [0.229,\ 0.224,\ 0.225]$$

- Resize to 224×224 (for `ResNet`, `EfficientNet`, `ViT` families)
- Apply the same normalisation to *target domain* images — a common mistake in Challenge 7 is forgetting this step for cartoon/sketch images
- Training augmentations: *random horizontal flip*, *random crop*, *color jitter*
- Validation/test: deterministic *centre crop* at 224×224; no augmentation

Challenge 7 Part A Connection

Few-Shot Classification on `DomainNet` (Part A)

- **Budget**: only 50 labelled images/class $\times$ 6 classes $= 300$ training images
- **Backbone**: `ResNet-50` with `ImageNet` weights
- **Strategies**: frozen backbone vs. fine-tuning vs. *from-scratch baseline*
- **Domains**: each group assigned a *source* $\rightarrow$ *target* pair (*Real*, *Clipart*, *Sketch*, *Painting*)

Expected Outcome

From-scratch will severely underfit. Frozen backbone will already achieve strong accuracy. Fine-tuning (if done *carefully*) gives the best result. The gap demonstrates the value of pretraining in *low-data regimes*.

Challenge 7 Part A Connection

Few-Shot Classification on `DomainNet` (Part A)

- **Budget**: only 50 labelled images/class $\times$ 6 classes $= 300$ training images
- **Backbone**: `ResNet-50` with `ImageNet` weights
- **Strategies**: frozen backbone vs. fine-tuning vs. *from-scratch baseline*
- **Domains**: each group assigned a *source* $\rightarrow$ *target* pair (*Real*, *Clipart*, *Sketch*, *Painting*)

Expected Outcome

From-scratch will severely underfit. Frozen backbone will already achieve strong accuracy. Fine-tuning (if done *carefully*) gives the best result. The gap demonstrates the value of pretraining in *low-data regimes*.

Practical Example: ResNet-50 Feature Extraction and Fine-Tuning

- **Setup**: DomainNet "Real" *photos*, 6 animal categories, 50 images/class
- **Run A**: frozen backbone + *linear head*; train 30 epochs, $\mathtt{LR} = 10^{-3}$
- **Run B**: unfreeze layer4 + head; train 30 epochs, $\mathtt{LR} = 10^{-4}$
- **Run C**: full fine-tune; $\mathtt{LR} = 5\times10^{-5}$; risk of overfitting
- **Metrics**: top-1 accuracy on *source test set*; confusion matrix
- **Visualisation**: t-SNE of backbone features — do classes separate even without *fine-tuning*?

Practical Example: ResNet-50 Feature Extraction and Fine-Tuning

- **Setup**: DomainNet "Real" *photos*, 6 animal categories, 50 images/class
- **Run A**: frozen backbone + *linear head*; train 30 epochs, LR $= 10^{-3}$
- **Run B**: unfreeze layer4 + head; train 30 epochs, LR $= 10^{-4}$
- **Run C**: full fine-tune; LR $= 5\times10^{-5}$; risk of overfitting
- **Metrics**: top-1 accuracy on *source test set*; confusion matrix
- **Visualisation**: t-SNE of backbone features — do classes separate even without *fine-tuning*?

Practical Example: ResNet-50 Feature Extraction and Fine-Tuning

- **Setup**: DomainNet "Real" *photos*, 6 animal categories, 50 images/class
- **Run A**: frozen backbone + *linear head*; train 30 epochs, LR $= 10^{-3}$
- **Run B**: unfreeze layer4 + head; train 30 epochs, LR $= 10^{-4}$
- **Run C**: full fine-tune; LR $= 5\times10^{-5}$; risk of overfitting
- **Metrics**: top-1 accuracy on *source test set*; confusion matrix
- **Visualisation**: t-SNE of backbone features — do classes separate even without *fine-tuning*?

Outline

1. Generative Adversarial Networks I
2. Generative Adversarial Networks II
3. Transfer Learning I: Pretrained Models & Few-Shot Classification
4. Transfer Learning II: Neural Style Transfer & Domain Adaptation
5. Online & Continual Learning

Neural Style Transfer

Gatys, Ecker & Bethge, 2015

Synthesise a new image that has the content of one image and the style of another — without training any *network*. We optimise the pixel values of the generated image.

- **Content**: encoded in deep VGG-19 *feature maps* (`relu4_2`)
- **Style**: encoded in Gram matrices of *shallow-to-mid* feature maps (`relu1_1`, `relu2_1`, `relu3_1`, `relu4_1`)
- Use a frozen VGG-19; the *network* is a differentiable *feature extractor* only

Neural Style Transfer

Gatys, Ecker & Bethge, 2015

Synthesise a new image that has the content of one image and the style of another — without training any *network*. We optimise the pixel values of the generated image.

- **Content**: encoded in deep `VGG-19` *feature maps* (`relu4_2`)
- **Style**: encoded in Gram matrices of *shallow-to-mid* feature maps (`relu1_1`, `relu2_1`, `relu3_1`, `relu4_1`)
- Use a frozen `VGG-19`; the *network* is a differentiable *feature extractor* only

Neural Style Transfer

Gatys, Ecker & Bethge, 2015

Synthesise a new image that has the content of one image and the style of another — without training any *network*. We optimise the pixel values of the generated image.

- **Content**: encoded in deep `VGG-19` *feature maps* (`relu4_2`)
- **Style**: encoded in Gram matrices of *shallow-to-mid* feature maps (`relu1_1`, `relu2_1`, `relu3_1`, `relu4_1`)
- Use a frozen `VGG-19`; the *network* is a differentiable *feature extractor* only

Gram Matrix: Encoding Style

Definition

For *feature map* $F^l \in \mathbb{R}^{C_l \times H_l W_l}$, the Gram matrix is:

$$G^l_{ij} = \frac{1}{C_l H_l W_l} \sum_k F^l_{ik} F^l_{jk}$$

- Captures pairwise correlations between feature channels
- Insensitive to spatial layout — it is a pure texture descriptor
- *Style loss:* $\mathcal{L}_{\text{style}} = \sum_l w_l \|G^l - \hat{G}^l\|_F^2$ where $\hat{G}^l$ is the target style image's Gram matrix

Gram Matrix: Encoding Style

Definition

For *feature map* $F^l \in \mathbb{R}^{C_l \times H_l W_l}$, the Gram matrix is:

$$G^l_{ij} = \frac{1}{C_l H_l W_l} \sum_k F^l_{ik} F^l_{jk}$$

- Captures pairwise correlations between feature channels
- Insensitive to spatial layout — it is a pure texture descriptor
- *Style loss*: $\mathcal{L}_{\text{style}} = \sum_l w_l \|G^l - \hat{G}^l\|_F^2$ where $\hat{G}^l$ is the target style *image's Gram matrix*

Gram Matrix: Encoding Style

Definition

For *feature map* $F^l \in \mathbb{R}^{C_l \times H_l W_l}$, the Gram matrix is:

$$G^l_{ij} = \frac{1}{C_l H_l W_l} \sum_k F^l_{ik} F^l_{jk}$$

- Captures pairwise correlations between feature channels
- Insensitive to spatial layout — it is a pure texture descriptor
- *Style loss*: $\mathcal{L}_{\text{style}} = \sum_l w_l \|G^l - \hat{G}^l\|_F^2$ where $\hat{G}^l$ is the target style *image's Gram matrix*

Style Transfer Total Loss and Optimisation

Total Loss

$$\mathcal{L}_{\text{total}} = \alpha\,\mathcal{L}_{\text{content}} + \beta\,\mathcal{L}_{\text{style}}$$

$$\mathcal{L}_{\text{content}} = \frac{1}{2}\sum_{l} \|F^l - P^l\|^2$$

- Typical: $\alpha = 1$, $\beta = 10^3$–10^6 (*style usually dominates*)
- **Optimiser**: L-BFGS (*quasi-Newton*) converges in ~300 iterations
- **Initialisation**: start from the content image for stable results

Style Transfer Total Loss and Optimisation

Total Loss

$$\mathcal{L}_{\text{total}} = \alpha\,\mathcal{L}_{\text{content}} + \beta\,\mathcal{L}_{\text{style}}$$

$$\mathcal{L}_{\text{content}} = \frac{1}{2}\sum_{l} \|F^l - P^l\|^2$$

- Typical: $\alpha = 1$, $\beta = 10^3$–10^6 (*style usually dominates*)
- **Optimiser**: `L-BFGS` (*quasi-Newton*) converges in ∼300 iterations
- **Initialisation**: start from the content image for stable results
- **Output**: a new image with *content* + *style*; directly serves as synthetic target-domain data in Challenge 7 Part C

Style Transfer Total Loss and Optimisation

Total Loss

$$\mathcal{L}_{\text{total}} = \alpha\, \mathcal{L}_{\text{content}} + \beta\, \mathcal{L}_{\text{style}}$$

$$\mathcal{L}_{\text{content}} = \frac{1}{2} \sum_{l} \|F^l - P^l\|^2$$

- Typical: $\alpha = 1$, $\beta = 10^3$–10^6 (*style usually dominates*)
- **Optimiser**: `L-BFGS` (*quasi-Newton*) converges in ∼300 iterations
- **Initialisation**: start from the content image for stable results
- **Output**: a new image with *content* + *style*; directly serves as synthetic target-domain data in Challenge 7 Part C

Style Transfer Total Loss and Optimisation

Total Loss

$$\mathcal{L}_{\text{total}} = \alpha\, \mathcal{L}_{\text{content}} + \beta\, \mathcal{L}_{\text{style}}$$

$$\mathcal{L}_{\text{content}} = \frac{1}{2} \sum_{l} \|F^l - P^l\|^2$$

- Typical: $\alpha = 1$, $\beta = 10^3$–10^6 (*style usually dominates*)
- **Optimiser**: `L-BFGS` (*quasi-Newton*) converges in ∼300 iterations
- **Initialisation**: start from the content image for stable results
- **Output**: a new image with *content* + *style*; directly serves as synthetic target-domain data in Challenge 7 Part C

Domain Shift

Definition

Domain shift (covariate shift) occurs when the input distribution changes between *training* (source) and *deployment* (target):
$p_S(\mathbf{x}) \neq p_T(\mathbf{x})$ yet $p_S(y|\mathbf{x}) \approx p_T(y|\mathbf{x})$

- **Domain shift penalty**: $\Delta_{\text{shift}} = \text{Acc}_{\text{source}} - \text{Acc}_{\text{target}}$
- Real examples: *scanner A* vs. *scanner B* (medical imaging), *daylight* vs. *night* (autonomous driving), *simulation* vs. *real robot*
- **Challenge 7**: *Real photos* → *Clipart / Sketch / Painting* — a clean controlled domain shift scenario
- Measure Δ_{shift} for both the *frozen backbone* and *fine-tuned model*

Domain Shift

Definition

Domain shift (covariate shift) occurs when the input distribution changes between *training* (source) and *deployment* (target):
$p_S(\mathbf{x}) \neq p_T(\mathbf{x})$ yet $p_S(y|\mathbf{x}) \approx p_T(y|\mathbf{x})$

- **Domain shift penalty**: $\Delta_{\text{shift}} = \text{Acc}_{\text{source}} - \text{Acc}_{\text{target}}$
- Real examples: *scanner A* vs. *scanner B* (medical imaging), *daylight* vs. *night* (autonomous driving), *simulation* vs. *real robot*
- **Challenge 7**: *Real photos* → *Clipart* / *Sketch* / *Painting* — a clean controlled domain shift scenario
- Measure Δ_{shift} for both the *frozen backbone* and *fine-tuned model*

Domain Shift

Definition

Domain shift (covariate shift) occurs when the input distribution changes between *training* (source) and *deployment* (target):
$p_S(\mathbf{x}) \neq p_T(\mathbf{x})$ yet $p_S(y|\mathbf{x}) \approx p_T(y|\mathbf{x})$

- **Domain shift penalty**: $\Delta_{\text{shift}} = \text{Acc}_{\text{source}} - \text{Acc}_{\text{target}}$
- Real examples: *scanner A* vs. *scanner B* (medical imaging), *daylight* vs. *night* (autonomous driving), *simulation* vs. *real robot*
- **Challenge 7**: *Real photos* $\rightarrow$ *Clipart* / *Sketch* / *Painting* — a clean controlled domain shift scenario
- Measure Δ_{shift} for both the *frozen backbone* and *fine-tuned model*

Three Adaptation Strategies I (Challenge 7 Part C)

Baseline
No adaptation

$\Delta_{\text{shift}} = \text{large}$

Target Fine-tuning
Few target labels

$\Delta_{\text{shift}} = \text{medium}$

Synthetic Data
Style transfer + retrain

$\Delta_{\text{shift}} = \text{small}$

Three Adaptation Strategies II (Challenge 7 Part C)

1. **Baseline** (*no adaptation*): apply the Part A source-trained classifier directly to target domain; measure Δ_{shift}
2. Target fine-tuning with *few labels*: *fine-tune* the *top layers* using a small set of labelled *target-domain* examples ($\sim$50/class); compare with Part A *fine-tuning*
3. Synthetic data augmentation (using Part B output): mix style-transferred images into the *source training set*; *retrain* and evaluate on *target domain*

Expected Ranking

Synthetic augmentation $\geq$ *Target fine-tuning* $\gg$ *No adaptation* (though results vary by *group* and *domain pair*)

Three Adaptation Strategies II (Challenge 7 Part C)

1. **Baseline** (*no adaptation*): apply the Part A source-trained classifier directly to target domain; measure Δ_{shift}
2. Target fine-tuning with *few labels*: *fine-tune* the *top layers* using a small set of labelled *target-domain* examples ($\sim$50/class); compare with Part A *fine-tuning*
3. Synthetic data augmentation (using Part B output): mix style-transferred images into the *source training set*; *retrain* and evaluate on *target domain*

Expected Ranking

Synthetic augmentation $\geq$ *Target fine-tuning* $\gg$ *No adaptation* (though results vary by *group* and *domain pair*)

Three Adaptation Strategies II (Challenge 7 Part C)

1. **Baseline** (*no adaptation*): apply the Part A source-trained classifier directly to target domain; measure Δ_{shift}
2. Target fine-tuning with *few labels*: *fine-tune* the *top layers* using a small set of labelled *target-domain* examples ($\sim$50/class); compare with Part A *fine-tuning*
3. Synthetic data augmentation (using Part B output): mix style-transferred images into the *source training set*; *retrain* and evaluate on *target domain*

Expected Ranking

Synthetic augmentation $\geq$ *Target fine-tuning* $\gg$ *No adaptation* (though results vary by *group* and *domain pair*)

Three Adaptation Strategies II (Challenge 7 Part C)

1. **Baseline** (*no adaptation*): apply the Part A source-trained classifier directly to target domain; measure Δ_{shift}
2. Target fine-tuning with *few labels*: *fine-tune* the *top layers* using a small set of labelled *target-domain* examples ($\sim$50/class); compare with Part A *fine-tuning*
3. Synthetic data augmentation (using Part B output): mix style-transferred images into the *source training set*; *retrain* and evaluate on *target domain*

Expected Ranking

Synthetic augmentation $\geq$ *Target fine-tuning* $\gg$ *No adaptation* (though results vary by *group* and *domain pair*)

Maximum Mean Discrepancy (MMD)

Measuring Domain Shift in Feature Space

$$\text{MMD}^2(p, q) = \|\mathbb{E}_{\mathbf{x}\sim p}[\phi(\mathbf{x})] - \mathbb{E}_{\mathbf{y}\sim q}[\phi(\mathbf{y})]\|^2_{\mathcal{H}}$$

where ϕ maps to a *Reproducing Kernel Hilbert Space* (typically `Gaussian RBF` kernel).

- Compute `MMD` between *source* and *target* feature embeddings from the backbone
- High `MMD`: *hard domain shift*; adaptation more difficult
- Track `MMD` before and after adaptation to quantify the alignment improvement
- Complement Δ_{shift} (*accuracy-based*) with `MMD` (*distribution-based*)

Maximum Mean Discrepancy (MMD)

Measuring Domain Shift in Feature Space

$$\text{MMD}^2(p, q) = \|\mathbb{E}_{\mathbf{x}\sim p}[\phi(\mathbf{x})] - \mathbb{E}_{\mathbf{y}\sim q}[\phi(\mathbf{y})]\|^2_{\mathcal{H}}$$

where ϕ maps to a *Reproducing Kernel Hilbert Space* (typically Gaussian RBF kernel).

- Compute MMD between *source* and *target* feature embeddings from the backbone
- High MMD: *hard domain shift*; adaptation more difficult
- Track MMD before and after adaptation to quantify the alignment improvement
- Complement Δ_{shift} (*accuracy-based*) with MMD (*distribution-based*)

Maximum Mean Discrepancy (MMD)

Measuring Domain Shift in Feature Space

$$\text{MMD}^2(p, q) = \|\mathbb{E}_{\mathbf{x}\sim p}[\phi(\mathbf{x})] - \mathbb{E}_{\mathbf{y}\sim q}[\phi(\mathbf{y})]\|_{\mathcal{H}}^2$$

where ϕ maps to a *Reproducing Kernel Hilbert Space* (typically Gaussian RBF kernel).

- Compute MMD between *source* and *target* feature embeddings from the backbone
- High MMD: *hard domain shift*; adaptation more difficult
- Track MMD before and after adaptation to quantify the alignment improvement
- Complement Δ_{shift} (*accuracy-based*) with MMD (*distribution-based*)

Domain-Adversarial Neural Networks (DANN)

Ganin *et al.*, 2016 — Gradient Reversal

Add a domain classifier head and a gradient reversal layer (GRL). During *backprop*, the GRL multiplies the gradient by $-\lambda$, making the *feature extractor* domain-invariant.

Combined Loss

$$\mathcal{L} = \mathcal{L}_{\text{class}} - \lambda \, \mathcal{L}_{\text{domain}}$$

- $\mathcal{L}_{\text{class}}$: *cross-entropy* on *labelled source* samples
- $\mathcal{L}_{\text{domain}}$: binary CE on source/target *domain labels* (0/1)
- GRL forces features that confuse the *domain classifier* $\Rightarrow$ domain-invariant
- **Optional** in Challenge 7; recommended for groups seeking *distinc*

Domain-Adversarial Neural Networks (DANN)

Ganin *et al.*, 2016 — Gradient Reversal

Add a domain classifier head and a gradient reversal layer (GRL). During *backprop*, the GRL multiplies the gradient by $-\lambda$, making the *feature extractor* domain-invariant.

Combined Loss

$$\mathcal{L} = \mathcal{L}_{\text{class}} - \lambda\, \mathcal{L}_{\text{domain}}$$

- $\mathcal{L}_{\text{class}}$: *cross-entropy* on *labelled source* samples
- $\mathcal{L}_{\text{domain}}$: binary CE on source/target *domain labels* (0/1)
- GRL forces features that confuse the *domain classifier* $\Rightarrow$ domain-invariant
- **Optional** in Challenge 7; recommended for groups seeking *distinc*

Domain-Adversarial Neural Networks (DANN)

Ganin *et al.*, 2016 — Gradient Reversal

Add a domain classifier head and a gradient reversal layer (GRL). During *backprop*, the GRL multiplies the gradient by $-\lambda$, making the *feature extractor* domain-invariant.

Combined Loss

$$\mathcal{L} = \mathcal{L}_{\text{class}} - \lambda\, \mathcal{L}_{\text{domain}}$$

- $\mathcal{L}_{\text{class}}$: *cross-entropy* on *labelled source* samples
- $\mathcal{L}_{\text{domain}}$: binary CE on source/target *domain labels* (0/1)
- GRL forces features that confuse the *domain classifier* $\Rightarrow$ domain-invariant
- **Optional** in Challenge 7; recommended for groups seeking *distinction*

Practical Example: Challenge 7 Full Pipeline

- **Data**: DomainNet Real (*source*) → Clipart (*target*), 6 classes
- **Part A**: frozen ResNet-50 on 50 img/class; accuracy on source *test set*
- **Part B**: Style-transfer 30 images/class with target-domain *style*
- **Part C**: evaluate *source model* on *target*; record Δ_{shift}
- **Adaptation**: retrain with synthetic data; compare to no-adaptation baseline
- **Visualise**: UMAP of source and target *features* before and *after adaptation* (*source* = blue, *target* = red); clusters should mix *after adaptation*

Practical Example: Challenge 7 Full Pipeline

- **Data**: `DomainNet` Real (*source*) → Clipart (*target*), 6 classes
- **Part A**: frozen `ResNet-50` on 50 img/class; accuracy on source *test set*
- **Part B**: Style-transfer 30 images/class with target-domain *style*
- **Part C**: evaluate *source model* on *target*; record Δ_{shift}
- **Adaptation**: retrain with synthetic data; compare to no-adaptation baseline
- **Visualise**: `UMAP` of source and target *features* before and *after* adaptation (*source* = blue, *target* = red); clusters should mix *after adaptation*

Practical Example: Challenge 7 Full Pipeline

- **Data**: `DomainNet` Real (*source*) $\rightarrow$ Clipart (*target*), 6 classes
- **Part A**: frozen `ResNet-50` on 50 img/class; accuracy on source *test set*
- **Part B**: Style-transfer 30 images/class with target-domain *style*
- **Part C**: evaluate *source model* on *target*; record Δ_{shift}
- **Adaptation**: retrain with synthetic data; compare to no-adaptation baseline
- **Visualise**: `UMAP` of source and target *features* before and *after* adaptation (*source* = blue, *target* = red); clusters should mix *after adaptation*

Outline

1. Generative Adversarial Networks I
2. Generative Adversarial Networks II
3. Transfer Learning I: Pretrained Models & Few-Shot Classification
4. Transfer Learning II: Neural Style Transfer & Domain Adaptation
5. Online & Continual Learning

The Online Learning Paradigm

Stream vs. *Batch*

In online learning, data arrives as a stream: each *sample* (or *mini-batch*) is seen once, then *discarded*. The model must adapt without storing all past data.

- **Motivation**: *financial fraud detection, sensor streams, recommendation systems, robotics* with a *continuously changing environment*
- **Key constraint**: memory budget $\ll$ full training set
- **Key challenge**: distribution shift over time; older observations may no longer reflect the current distribution

The Online Learning Paradigm

Stream vs. *Batch*

In online learning, data arrives as a stream: each *sample* (or *mini-batch*) is seen once, then *discarded*. The model must adapt without storing all past data.

- **Motivation**: *financial fraud detection*, *sensor streams*, *recommendation systems*, *robotics* with a *continuously changing environment*
- **Key constraint**: memory budget $\ll$ full training set
- **Key challenge**: distribution shift over time; older observations may no longer reflect the current distribution

The Online Learning Paradigm

Stream vs. *Batch*

In online learning, data arrives as a stream: each *sample* (or *mini-batch*) is seen once, then *discarded*. The model must adapt without storing all past data.

- **Motivation**: *financial fraud detection*, *sensor streams*, *recommendation systems*, *robotics* with a *continuously changing environment*
- **Key constraint**: memory budget $\ll$ full training set
- **Key challenge**: distribution shift over time; older observations may no longer reflect the current distribution

Catastrophic Forgetting

McCloskey & Cohen, 1989 — The `Stability--Plasticity` Dilemma

When a neural network is trained sequentially on *Task A*, then *Task B*, it forgets *Task A*. New gradient updates overwrite weights critical for *Task A*.

- **Example**: a `ResNet` trained on *Cats/Dogs*, then retrained on *Cars*, will score near-chance on *Dogs*
- The network must be simultaneously stable (*retain old knowledge*) and plastic (*acquire new knowledge*)
- **Connection to** RL: switching `Atari` games between Challenges 1, 3, 4 triggers catastrophic forgetting if the same network is reused

Catastrophic Forgetting

McCloskey & Cohen, 1989 — The `Stability--Plasticity` Dilemma

When a neural network is trained sequentially on *Task A*, then *Task B*, it forgets *Task A*. New gradient updates overwrite weights critical for *Task A*.

- **Example**: a `ResNet` trained on *Cats/Dogs*, then retrained on *Cars*, will score near-chance on *Dogs*
- The network must be simultaneously stable (*retain old knowledge*) and plastic (*acquire new knowledge*)
- **Connection to** `RL`: switching `Atari` games between Challenges 1, 3, 4 triggers catastrophic forgetting if the same network is reused

Catastrophic Forgetting

McCloskey & Cohen, 1989 — The `Stability--Plasticity` Dilemma

When a neural network is trained sequentially on *Task A*, then *Task B*, it forgets *Task A*. New gradient updates overwrite weights critical for *Task A*.

- **Example**: a `ResNet` trained on *Cats/Dogs*, then retrained on *Cars*, will score near-chance on *Dogs*
- The network must be simultaneously stable (*retain old knowledge*) and plastic (*acquire new knowledge*)
- **Connection to** `RL`: switching `Atari` games between Challenges 1, 3, 4 triggers catastrophic forgetting if the same network is reused

Three Approaches to Continual Learning

Regularisation

- Penalise changes to important weights
- EWC, SI, MAS
- No extra memory for *data*

Replay

- Store or generate past examples
- Mix into current *training*
- ER, DER, Generative Replay

Architecture

- Allocate new capacity per *task*
- No forgetting by *design*
- PNN, PackNet
- Grows with *tasks*

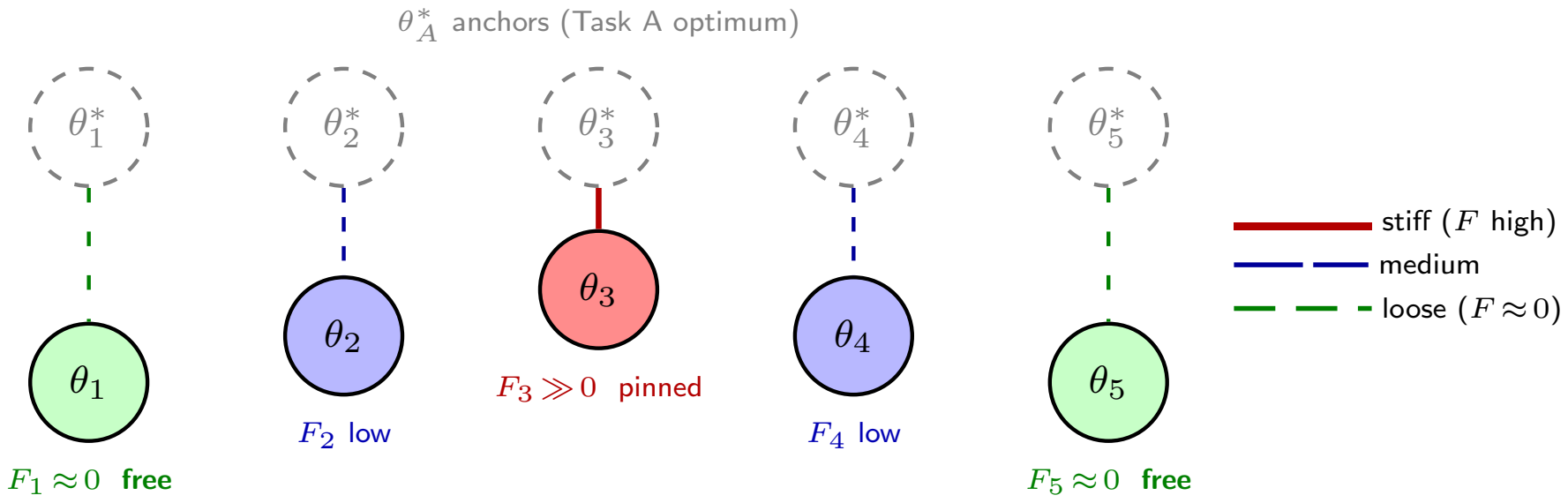
Elastic Weight Consolidation (EWC) I

Kirkpatrick *et al.*, 2017

After *Task A*, compute the Fisher Information F_i for each weight θ_i: F_i measures how much θ_i contributed to *Task A* performance. Then regularise training on *Task B*:

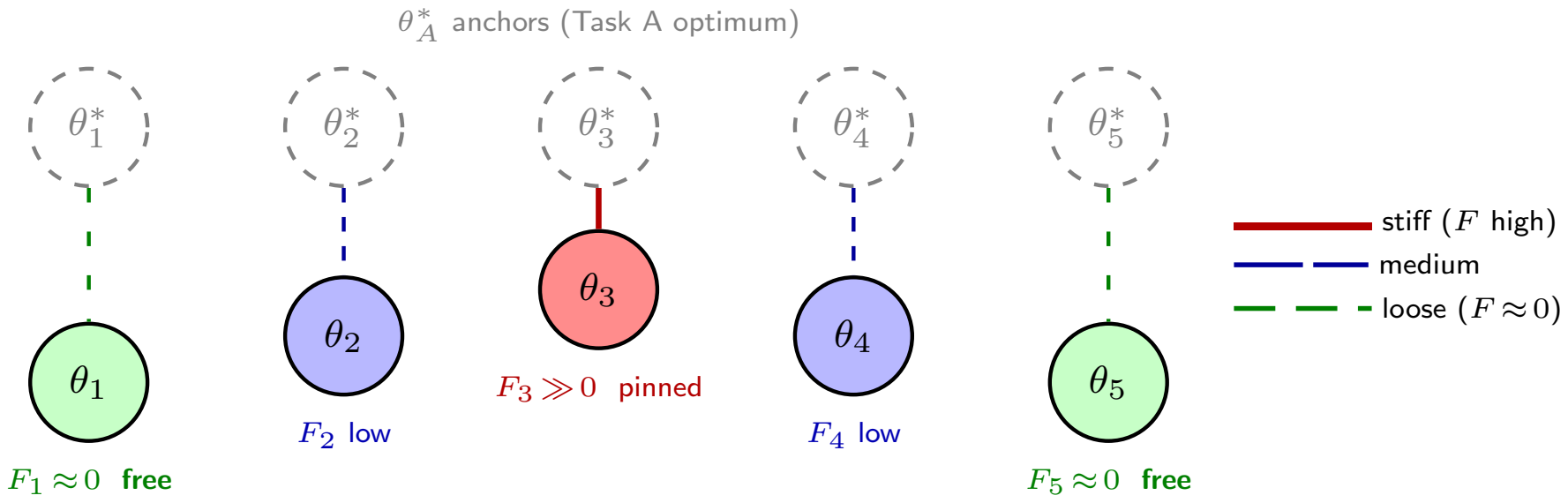
$$\mathcal{L}_{\text{EWC}} = \mathcal{L}_B(\theta) + \frac{\lambda}{2} \sum_i F_i \left(\theta_i - \theta^*_{A,i}\right)^2$$

Elastic Weight Consolidation (EWC) II



- θ_A^*: *optimal weights* after *Task A*
- High $F_i \Rightarrow$ important for Task A $\Rightarrow$ highly constrained
- **Approximation**: *diagonal* of the `Fisher` matrix (*per-weight importance*)
- Effective for a *small number* of sequential tasks; degrades as tasks accumulate

Elastic Weight Consolidation (EWC) II



- θ_A^*: *optimal weights* after *Task A*
- High $F_i \Rightarrow$ important for Task A $\Rightarrow$ highly constrained
- **Approximation**: *diagonal* of the `Fisher` matrix (*per-weight importance*)
- Effective for a *small number* of sequential tasks; degrades as tasks accumulate

Experience Replay

Core Idea

Maintain an episodic memory buffer $\mathcal{M}$ of *past examples*. When training on new data, interleave replayed examples:

$$\mathcal{L} = \mathcal{L}_{\text{new}}(\theta) + \alpha\, \mathcal{L}_{\text{replay}}(\theta; \mathcal{M})$$

- **Reservoir sampling**: maintains a *uniform random sample* of past data as new data arrives
- DER (*Dark Experience Replay*): replay soft logits from the past checkpoint (knowledge distillation); more informative than *hard labels*
- **Generative replay**: use a GAN or VAE to *synthesise* past experiences — no raw data storage needed
- **Memory buffer size** is the key *computational trade-off*

Experience Replay

Core Idea

Maintain an episodic memory buffer $\mathcal{M}$ of *past examples*. When training on new data, interleave replayed examples:

$$\mathcal{L} = \mathcal{L}_{\text{new}}(\theta) + \alpha\, \mathcal{L}_{\text{replay}}(\theta; \mathcal{M})$$

- **Reservoir sampling**: maintains a *uniform random sample* of past data as new data arrives
- DER (*Dark Experience Replay*): replay soft logits from the past checkpoint (knowledge distillation); more informative than *hard labels*
- **Generative replay**: use a GAN or VAE to *synthesise* past experiences — no raw data storage needed
- **Memory buffer size** is the key *computational trade-off*

Experience Replay

Core Idea

Maintain an episodic memory buffer $\mathcal{M}$ of *past examples*. When training on new data, interleave replayed examples:

$$\mathcal{L} = \mathcal{L}_{\text{new}}(\theta) + \alpha\, \mathcal{L}_{\text{replay}}(\theta; \mathcal{M})$$

- **Reservoir sampling**: maintains a *uniform random sample* of past data as new data arrives
- `DER` (*Dark Experience Replay*): replay soft logits from the past checkpoint (knowledge distillation); more informative than *hard labels*
- **Generative replay**: use a `GAN` or `VAE` to *synthesise* past experiences — no raw data storage needed
- **Memory buffer size** is the key *computational trade-off*

Self-Supervised and Contrastive Learning

Pretext Tasks

Create free supervision from the data itself: predict *rotation angle*, reconstruct *masked patches* (`MAE`), solve *jigsaw puzzles*.

`SimCLR` (Chen *et al.*, 2020)

Apply two *random augmentations* to the same image; pull the two views together, push views of different images apart:

$$\mathcal{L}_{\text{NT-Xent}} = -\log \frac{\exp(\text{sim}(\mathbf{z}_i, \mathbf{z}_j)/\tau)}{\sum_{k \neq i} \exp(\text{sim}(\mathbf{z}_i, \mathbf{z}_k)/\tau)}$$

- *Self-supervised* pre-training + linear probe: competitive with supervised pre-training
- Enables transfer learning without any labels in pretraining
- Foundation models: CLIP, DINOv2, MAE, SAM

Self-Supervised and Contrastive Learning

Pretext Tasks

Create free supervision from the data itself: predict *rotation angle*, reconstruct *masked patches* (`MAE`), solve *jigsaw puzzles*.

`SimCLR` (Chen *et al.*, 2020)

Apply two *random augmentations* to the same image; pull the two views together, push views of different images apart:

$$\mathcal{L}_{\text{NT-Xent}} = -\log \frac{\exp(\text{sim}(\mathbf{z}_i, \mathbf{z}_j)/\tau)}{\sum_{k \neq i} \exp(\text{sim}(\mathbf{z}_i, \mathbf{z}_k)/\tau)}$$

- *Self-supervised* pre-training + linear probe: competitive with supervised pre-training
- Enables transfer learning without any labels in pretraining
- **Connection:** CLIP uses contrastive learning across *image–text pairs*

Self-Supervised and Contrastive Learning

Pretext Tasks

Create free supervision from the data itself: predict *rotation angle*, reconstruct *masked patches* (`MAE`), solve *jigsaw puzzles*.

`SimCLR` (Chen *et al.*, 2020)

Apply two *random augmentations* to the same image; pull the two views together, push views of different images apart:

$$\mathcal{L}_{\text{NT-Xent}} = -\log \frac{\exp(\text{sim}(\mathbf{z}_i, \mathbf{z}_j)/\tau)}{\sum_{k \neq i} \exp(\text{sim}(\mathbf{z}_i, \mathbf{z}_k)/\tau)}$$

- *Self-supervised* pre-training + linear probe: competitive with supervised pre-training
- Enables transfer learning without any labels in pretraining

Connection: `CLIP` uses contrastive learning across *image–text pairs*

Self-Supervised and Contrastive Learning

Pretext Tasks

Create free supervision from the data itself: predict *rotation angle*, reconstruct *masked patches* (`MAE`), solve *jigsaw puzzles*.

`SimCLR` (Chen *et al.*, 2020)

Apply two *random augmentations* to the same image; pull the two views together, push views of different images apart:

$$\mathcal{L}_{\text{NT-Xent}} = -\log \frac{\exp(\text{sim}(\mathbf{z}_i, \mathbf{z}_j)/\tau)}{\sum_{k \neq i} \exp(\text{sim}(\mathbf{z}_i, \mathbf{z}_k)/\tau)}$$

- *Self-supervised* pre-training + linear probe: competitive with supervised pre-training
- Enables transfer learning without any labels in pretraining

Connection: `CLIP` uses contrastive learning across *image–text pairs*

Practical Example: EWC on Sequential MNIST variants

- **Task A**: classify digits 0–4 on standard MNIST
- **Task B**: classify digits 5–9 on permuted MNIST (*pixels shuffled*)
- **Baseline**: *sequential fine-tuning* — observe sharp accuracy drop on *Task A*
- EWC: add Fisher-weighted regularisation; measure *retention*
- **Replay**: compare EWC with a 500-sample *memory buffer*
- **Expected**: Replay $\geq$ EWC $\gg$ naive *sequential fine-tuning*
- **Metric**: backward transfer $= \frac{1}{T-1}\sum_{i<T}(A_{T,i} - A_{i,i})$ (*average accuracy drop* on prior tasks after training on all tasks)

Practical Example: EWC on Sequential MNIST variants

- **Task A**: classify digits 0–4 on standard MNIST
- **Task B**: classify digits 5–9 on permuted MNIST (*pixels shuffled*)
- **Baseline**: *sequential fine-tuning* — observe sharp accuracy drop on *Task A*
- EWC: add Fisher-weighted regularisation; measure *retention*
- **Replay**: compare EWC with a 500-sample *memory buffer*
- **Expected**: Replay $\geq$ EWC $\gg$ naive *sequential fine-tuning*
- **Metric**: backward transfer $= \frac{1}{T-1}\sum_{i<T}(A_{T,i} - A_{i,i})$ (*average accuracy drop* on prior tasks after training on all tasks)

Practical Example: EWC on Sequential MNIST variants

- **Task A**: classify digits 0–4 on standard MNIST
- **Task B**: classify digits 5–9 on permuted MNIST (*pixels shuffled*)
- **Baseline**: *sequential fine-tuning* — observe sharp accuracy drop on *Task A*
- EWC: add Fisher-weighted regularisation; measure *retention*
- **Replay**: compare EWC with a 500-sample *memory buffer*
- **Expected**: Replay $\geq$ EWC $\gg$ naive *sequential fine-tuning*
- **Metric**: backward transfer $= \frac{1}{T-1}\sum_{i<T}(A_{T,i} - A_{i,i})$ (*average accuracy drop* on prior tasks after training on all tasks)

Outline

1. Generative Adversarial Networks I
2. Generative Adversarial Networks II
3. Transfer Learning I: Pretrained Models & Few-Shot Classification
4. Transfer Learning II: Neural Style Transfer & Domain Adaptation
5. Online & Continual Learning

Thanks!

Questions?

Repo: *https://github.com/EngAndres/ud-public/tree/main/courses/machine-learning*